

JVM Model

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 2

Conventions . . . 3

I: The Why . . . 4

1. What your assembly became . . . 5

1.1. Declared is not running . . . 5

1.2. Code on the class path is visible too . . . 6

1.3. The missing view . . . 6

2. Identity . . . 7

2.1. Why the JVM's own names . . . 7

2.2. Joining is mechanical . . . 7

2.3. Erasure, and generics as data . . . 8

3. A spine, not a picture . . . 9

3.1. What is in, and what is out . . . 9

3.2. Why the JVM model does not extend Meta . . . 9

3.3. Three decisions . . . 10

II: The How . . . 11

4. Getting started . . . 12

4.1. Adding the model . . . 12

4.2. Names that clash . . . 12

4.3. What the build does . . . 13

5. The model . . . 14

5.1. JavaVirtualMachine . . . 14

5.2. Layer and ClassLoader . . . 14

5.3. Module and its directives . . . 15

5.4. Package and Type . . . 15

5.5. Members . . . 16

6. A snapshot of a running JVM . . . 18

6.1. Modules and readability . . . 18

6.2. Saving and loading . . . 19

6.3. Asking a question . . . 20

6.4. Types and methods, named the JVM's way . . . 21
III: Looking Ahead . . . **23**
7. Looking ahead . . . **24**
7.1. A live loader . . . 24
7.2. A scanner of class files . . . 24
7.3. Annotations, and the projection to Meta . . . 24
7.4. Applications . . . 25
7.5. Documentation, and a licensing constraint . . . 25
Appendix A: Reference . . . **27**
A.1. Classes . . . 27
A.2. Enumerations . . . 27
A.3. Identity . . . 28
Appendix B: Building this book . . . **29**
Appendix C: Glossary . . . **30**

Colophon

JVM Model: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the JVM model of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime.models.jvm:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/jvm`. Every listing in it is included from the module's tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *JVM Model: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Java and OpenJDK are trademarks or registered trademarks of Oracle and its affiliates. Eclipse and EMF are trademarks of the Eclipse Foundation. OpenTelemetry is a trademark of The Linux Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies the model relates to and implies no endorsement.

The source of this book, its tests and the model it describes are at github.com/Nasdanika-SDK/runtime, in `models/jvm/model`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about a model of the Java virtual machine: its layers, class loaders, modules, packages, types, methods, fields and parameters, as an EMF model that can be saved, compared, queried and published like any other. Its point is that a running JVM already knows what it is made of, and almost nobody asks it.

The first part of the book is the why: the gap between what a build declares and what runs, the naming that lets everything about a JVM join on one key, and what the model deliberately leaves to others. The second part is the how, for Java developers, and ends with a snapshot of a running module layer that is saved, loaded back and asked which modules read what they never required. The third says what comes next: the loaders, and what they make possible.

Who this book is for

Java developers and architects who want to know what an application became after resolution, not what its build files say, and who want to keep that answer.

Builders of tools for coverage, profiling, tracing and documentation, who need a key that their outputs and everyone else's share.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. You do not need any other book of the series, or the Nasdanika SDK Runtime book, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The model's Java module is `org.nasdanika.sdk.runtime.models.jvm`, its Maven coordinates are

`org.nasdanika.sdk.runtime.models.jvm:model:2026.11.0`, and its namespace URI is `runtime.sdk.nasdanika.org/models/jvm`.

- Some familiarity with EMF helps in part two: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.
- Some familiarity with the Java module system helps too: modules, `requires`, `exports`, `opens`, `provides` and `uses`.

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built, **SKETCH** for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the model and its documentation found while writing. They are shown in builds for the author and left out of published editions.



This book replaces the model's readme, `models/jvm/model/doc/readme.md`, which is deleted. The readme said that "the Nasdanika SDK Runtime book shows a minimal live loader of modules in a page of code"; this book has its own, in its tests. The comment on `JvmElement` in the Xcore still says "NxCore identity"; the runtime's name is the core model.

Part I: The Why

Why a running JVM should be a model, how its elements are named, and what the model leaves to others. No code is needed to read this part.

Chapter 1. What your assembly became

A running Java virtual machine knows a great deal about itself. `ModuleLayer.boot()` gives the module graph, `Configuration` reports what resolution actually produced, and reflection has exposed types and members since Java 1.1. The information was never the problem. Getting an answer meant writing a program, so only questions worth a program were ever asked, and that is a small set.

A typed model of the JVM, saved and queried, lowers the cost of a question to the cost of asking it. And because the model is data, the answer can be kept: next to the build that produced the JVM, in the same repository, compared with last month's.

1.1. Declared is not running

There is a gap between what a build declares and what runs, and nothing reports it. A dependency tree describes what could be on the path, test scope, optional dependencies and libraries that nothing ever loads included. The resolved module graph is shorter and truer.

Its clearest instance is readability: modules that read something they never required. Usually that is correct and by design:

- `requires transitive` propagates readability: a module that requires the JVM model reads the core model and EMF too.
- Service binding pulls providers into the resolved graph that no dependency declaration mentions.
- A module that reads one automatic module, a jar without a module descriptor, reads all of them.

All three are properties of resolution, not of the build, so they are absent from every POM and every bill of materials by construction, and no dependency tool can report them. A snapshot of the resolved graph is what your assembly became.

1.2. Code on the class path is visible too

Code on the class path lives in a class loader’s unnamed module. The unnamed module is a value, with packages and types in it, not an absence. So the model is as useful for an application that has never heard of the module system as for one that is modular throughout.

1.3. The missing view

Logs, metrics, traces and profiles describe behavior over time. None of them describes composition: which modules, which versions, which class file versions, which packages opened to whom. A model of the JVM is that view, and it is the one the others have been missing, because a span, a profile sample and a coverage line all point into it (Chapter 2).

Chapter 2. Identity

The decision everything else rests on is how elements are named. The JVM model names them the way the JVM does: binary names and descriptors, such as `java.util.List`, `java.util.Map$Entry`, `[I` and `get(I)Ljava/lang/Object;`.

2.1. Why the JVM's own names

That is not a local convention. It is the form in the constant pool, in stack traces, in `jdeps` output, in JDK Flight Recorder events, and in profilers and coverage tools. OpenTelemetry's code attributes carry its two halves: the type and the method. A build scan, a coverage report, a profile and a span name the same method the same way, so models produced independently merge on their URIs with no translation step:

Element	URI
Module	`jvm:module/<module name>`
Package	`jvm:package/<binary package name>`
Type	`jvm:type/<binary name>`
Method	`jvm:method/<owner binary name>#<name><descriptor>`
Field	`jvm:field/<owner binary name>#<name>:<descriptor>`
Parameter	`jvm:parameter/<owner method>#<index>`

`List.get`, for example, is `jvm:method/java.util.List#get(I)Ljava/lang/Object;`, whoever loaded it and whatever tool reported it.

2.2. Joining is mechanical

Joining static structure to runtime observation is then mechanical, not a project. JaCoCo is the worked example: its XML report already keys classes by binary name and methods by name and descriptor, so coverage attaches to the elements of a JVM model with no mapping layer and no configuration per project. A slow span resolves to a method, the method to a type in a module, the module to the artifact it came from, and the artifact to the team that owns it.

2.3. Erasure, and generics as data

Descriptors are erased: `List.get` returns `Ljava/lang/Object;`, not the element type. That is right for identity, because the JVM tells overloads apart by their erased descriptors, so two members of a type never differ only in their generic types.

Generic types are carried as data: the class file's `Signature` attribute, verbatim, on types and members, such as `<T:Ljava/lang/Object;>(TT;)Ljava/util/Optional<TT;>;`. It is the JVM's own form rather than Java source text, for the same reason identity uses descriptors: it is what bytecode carries, so a scan of class files copies it, and a live loader writes the same string from `java.lang.reflect.Type`. The difference between the two strings is also the difference between kinds of change: a change to a signature alone breaks source compatibility, and a change to a descriptor breaks binary compatibility. Parsing signatures into type structure is left to consumers, such as a projection to the runtime's Meta model.

Chapter 3. A spine, not a picture

The JVM model is a spine, not a complete picture of a JVM, and that is the design. Information about source, bytecode and run time attaches to its elements instead of restating them:

- a model of Java source adds positions, comments and the type text of the source;
- a model of bytecode adds instructions and constant pools;
- the runtime's telemetry model and coverage reports add measurements;

and all of them point at the same elements.

3.1. What is in, and what is out

	What
In	Layers, class loaders, modules and their directives, packages, types, methods, fields, parameters, resolved readability, access flags, class file versions
Generics as data	The class file's `Signature` attribute, verbatim, on types and members (Chapter 2)
Out	Source positions, instructions, constant pools, annotations
Projected, not extended	Meta, the runtime's language-neutral type system

Annotations are the most likely first addition **PLANNED**. They are among the most valuable things to query in a JVM estate, "every deprecated method", "every type with a given framework annotation", and they are an increment that disturbs nothing else.

3.2. Why the JVM model does not extend Meta

The runtime's Meta model covers packages, classes, features and operations, loads from Java among other formats, and is deliberately not a superset of any of its sources: whatever does not fit is dropped or carried in an annotation. That is a published language, and a published language is a target of projections, not a base class.

The test is concrete. Bridge methods, synthetic accessors, access flags and overloads that differ only in erased parameter types are realities of the JVM that Meta will not grow to hold. Extending

Meta would shape a model whose whole value is fidelity to the JVM's naming by one that is deliberately lossy about it. So the JVM model sits on the runtime's core model, and projects to Meta **PLANNED**, which gives anyone comparing a Java type with an Ecore class or a JSON Schema the route they expect.

3.3. Three decisions

Three decisions are worth knowing before reading the classes:

- **Modules are contained by the root, not by a layer.** A named module belongs to a layer and an unnamed module to a class loader, and one containment cannot express both, so layers and loaders reference modules.
- **Constructors are methods.** In the JVM a constructor is a method named `<init>` and a static initializer is `<clinit>`; descriptors tell overloads apart, and the naming works the same way for all three. Reflection separates constructors from methods, bytecode does not, and the model follows bytecode because that is where its identity comes from.
- **Access flags are booleans, not a packed integer.** A query language without bitwise operators could not ask about an integer of flags. Choices that exclude each other, such as the access level and the kind of a type, are enumerations.

Part II: The How

The model in Java: adding it to a project, its classes, and a snapshot of a running module layer that is saved, loaded and asked a question.

Chapter 4. Getting started

The model is an EMF model, defined in Xcore and generated into Java when the runtime is built.

4.1. Adding the model

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on the model without a version:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.jvm</groupId>
    <artifactId>model</artifactId>
</dependency>
```

The model is a named Java module:

Listing 1. The module declaration of the JVM model

```
module org.nasdanika.sdk.runtime.models.jvm {

    exports org.nasdanika.sdk.runtime.models.jvm;
    exports org.nasdanika.sdk.runtime.models.jvm.impl;
    exports org.nasdanika.sdk.runtime.models.jvm.util;

    requires transitive org.nasdanika.sdk.runtime.models.core;

    provides EPackage with JvmEPackageProvider;

}
```

It requires the runtime's core model transitively, which brings EMF with it, and provides its `EPackage` as a service, so a resource set configured from the module path knows the model.

4.2. Names that clash

The model's classes have the JVM's names: `Module`, `Package`, `Type`, `Method`, `Field`, `ClassLoader`. `Module`, `Package` and `ClassLoader` are also in `java.lang`, which every Java file imports. An explicit import of the model's class wins over `java.lang` in that file; the JDK's class is then written with its package, `java.lang.Module`, as in the listings of this book.

4.3. What the build does

The Xcore definition is in `src/main/resources/org/nasdanika/sdk/runtime/models/jvm/jvm.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generated API is in three packages: the interfaces in `org.nasdanika.sdk.runtime.models.jvm`, their implementations in `impl`, and the switch and adapter factory in `util`. The package provider is hand-written, in `services`.

Chapter 5. The model

Every element of the model extends `JvmElement`, which extends the core model's `NamedElement`. So every element has an `id`, a `name`, documentation, markers that record where it came from, and `uris`. The root, `JavaVirtualMachine`, extends the core model's `ModelElement`: it has no name of its own.

5.1. JavaVirtualMachine

One JVM, described:

`live`

: True when the model was populated from a running VM, false for a scan of class files. Whether anything was observed running matters enough to be in the data, not inferred.

`vmName`, `vmVersion`, `vmVendor`

: The `java.vm.name`, `java.vm.version` and `java.vm.vendor` properties, when known.

`specificationVersion`

: The Java SE specification version, such as `25`.

`modules`, `layers`, `classLoaders`

: Containments. Modules are contained here, not in their layers (Chapter 3).

5.2. Layer and ClassLoader

A `Layer` is a module layer, with `boot` true for the boot layer, its `parents` and its `modules`. Most applications have one layer, the boot layer, and the interesting case is the one that does not: layers are the only standard way to run two versions of a module in one JVM, which makes "which version is this code seeing" answerable where a class path makes it guesswork.

A `ClassLoader` has the binary name of its own class in `loaderTypeName`, its `parent`, and its `unnamedModule`. Loaders are orthogonal to layers: a loader may serve modules in several layers, and every loader has exactly one unnamed module, holding what it loaded from the class path.

5.3. Module and its directives

A `Module` has `named`, false for an unnamed module, the raw `version` string of its descriptor, `modifiers` (`OPEN`, `AUTOMATIC`, `SYNTHETIC`, `MANDATED`), and `mainClassName`. It contains its `packages` and its directives:

Class	Directive
`Requires`	`requires`, with `modifiers` (`TRANSITIVE`, `STATIC`, `SYNTHETIC`, `MANDATED`), the `compiledVersion`, and the required `module` when it is in the model. The name is authoritative; the reference is resolved when it can be.
`Exports`	`exports`, of a `package`, to `targets` and `targetNames`; empty targets mean an export to everyone
`Opens`	`opens`, the same shape as `Exports`. It is a separate class because the difference, deep reflective access instead of access at compile time, is the whole point of the directive
`Provides`	`provides`, with the `serviceName` and the `providerNames`, and references to the `service` and `providers` types

`uses` lists the binary names of the service interfaces the module uses.

Two references carry the reason the model is worth having. `requires` is what the module declared; `reads` is what resolution gave it, taken from the configuration, not from the descriptor. The difference between the two is the first question to ask of a snapshot (Chapter 6). `layer` and `classLoader` say where the module lives.

5.4. Package and Type

A `Package` has the dotted package name as its `name`, empty for the unnamed package, and contains its `types`.

A `Type` is a class, an interface, an enum, a record, an annotation, a primitive or an array, told apart by `kind`. It is one class, not a hierarchy, as in reflection, where `isPrimitive` and `isArray` are questions asked of `Class`; that keeps the naming uniform, since `int` and `int[]` have binary names like everything else.

`binaryName`, `descriptor`, `signature`

The identity-bearing name, such as `java.util.Map$Entry`; the descriptor, such as

`Ljava/util/Map$Entry;`; and the generic signature from the class file, unset when the type declares nothing generic.

`access`

`PUBLIC`, `PROTECTED`, `PACKAGE` or `PRIVATE`.

`abstract`, **`final`**, **`synthetic`**, **`sealed`**

Flags, as booleans.

`classFileVersion`

The major version of the class file, such as 69 for Java 25, or -1 when unknown, which includes primitives and arrays. Two bytes of the header that nothing aggregates, and the basis of a report about an estate that nobody has.

`methods`, **`fields`**

Containments.

`superType`, **`interfaces`**, **`componentType`**, **`module`**

References. `componentType` is set for arrays; `module` is a convenience, since the package is the container.

5.5. Members

`Member` is the abstract base of methods and fields, with `access`, `static`, `final`, `synthetic`, the `descriptor`, and the generic `signature`, unset when the descriptor says everything.

A `Method` is a method, a constructor (`<init>`) or a static initializer (`<clinit>`). It has the flags `abstract`, `synchronized`, `native`, `bridge`, `varargs` and `default`, the `returnDescriptor`, its `parameters`, its `returnType`, and its declared exceptions as `thrownTypeNames` and `thrownTypes`.

A `Field` has the flags `transient`, `volatile` and `enumConstant`, its `type`, and its `constantValue`. The constant value is API, not detail: javac copies a constant's value into every class that reads it, so client code carries the value it was compiled against, and changing it is a break that no comparison of descriptors shows. The text is the value as the class file stores it, read with the field's descriptor: booleans, bytes, characters, shorts and integers as decimal integers, since all five are stored as `int`; floats and doubles as `Float.toString` and `Double.toString` write them; strings

with Java escapes, so that the value survives XML. Reflection cannot provide it: a static final field initialized with a constant and one initialized in a static initializer look the same through `java.lang.reflect`. It comes from the class file, which a live loader can read too.

A `Parameter` has its `index`, its `descriptor`, `final`, `synthetic`, `mandated` for parameters the compiler generated, and its `type`. The `name` is present only when the class was compiled with `-parameters`, so the position is the reliable identity.

Chapter 6. A snapshot of a running JVM

The runtime's live loader is **PLANNED**. A minimal one fits in a page, and shows the shape; it is in this book's tests, as `Snapshot`.

6.1. Modules and readability

Listing 2. A snapshot of a module layer (`Snapshot.java`)

```
public static JavaVirtualMachine of(ModuleLayer layer) {
    JavaVirtualMachine vm = JVM.createJavaVirtualMachine();
    vm.setLive(true);                                                    ①
    vm.setVmName(System.getProperty("java.vm.name"));
    vm.setVmVersion(System.getProperty("java.vm.version"));
    vm.setSpecificationVersion(System.getProperty("java.specification.version"));

    Layer boot = JVM.createLayer();
    boot.setName("boot");
    boot.setBoot(true);
    vm.getLayers().add(boot);

    Map<String, Module> modules = new TreeMap<>();
    for (java.lang.Module running: layer.modules()) {                    ②
        ModuleDescriptor descriptor = running.getDescriptor();
        Module module = JVM.createModule();
        module.setName(running.getName());
        module.setId("jvm:module/" + running.getName());                 ③
        module.setNamed(true);
        descriptor.rawVersion().ifPresent(module::setVersion);
        if (descriptor.isAutomatic()) {
            module.getModifiers().add(ModuleModifier.AUTOMATIC);
        }
        module.getUses().addAll(new TreeSet<>(descriptor.uses()));
        for (ModuleDescriptor.Provides service: descriptor.provides()) {
            Provides provides = JVM.createProvides();
            provides.setServiceName(service.service());
            provides.getProviderNames().addAll(service.providers());
            module.getProvides().add(provides);
        }
        modules.put(module.getName(), module);
    }
    vm.getModules().addAll(modules.values());
    boot.getModules().addAll(modules.values());

    Configuration configuration = layer.configuration();
    for (Module module: modules.values()) {
        ModuleDescriptor descriptor = layer.findModule(module.getName())
                .orElseThrow()
                .getDescriptor();
        for (ModuleDescriptor.Requires declared: descriptor.requires()) {
```

```
            Requires requires = JVM.createRequires();
            requires.setName(declared.name());
            for (ModuleDescriptor.Requires.Modifier modifier: declared.modifiers()) {
                requires.getModifiers().add(RequiresModifier.valueOf(modifier.name()));
            }
            requires.setModule(modules.get(declared.name()));               ④
            module.getRequires().add(requires);
        }
        configuration.findModule(module.getName()).ifPresent(resolved -> {  ⑤
            for (ResolvedModule read: resolved.reads()) {
                Module target = modules.get(read.name());
                if (target != null) {
                    module.getReads().add(target);
                }
            }
        });
    }
    return vm;
}
```

① Observed in a running VM, not scanned from jars.

② What each module declares, from its descriptor.

③ The JVM's own naming, as the identity of the element.

④ A required module that was not resolved stays unresolved: the declaration is kept, with no target.

⑤ What resolution made of the declarations: the modules each module reads.

6.2. Saving and loading

A snapshot is a model, so it is saved as evidence of what ran, next to the build or the release it describes. The listing saves it as XMI with EMF alone; with the runtime's `common` module on the module path, `NasdanikaResourceSet.createAndConfigure()` picks the format from the extension instead, YAML and JSON among them.

Listing 3. Saving a snapshot (`SnapshotTests.java`)

```
JavaVirtualMachine vm = Snapshot.of(ModuleLayer.boot());

ResourceSet resourceSet = new ResourceSetImpl();
resourceSet.getResourceFactoryRegistry()
    .getExtensionToFactoryMap()
    .put("xml", new XMIResourceFactoryImpl());
URI uri = URI.createFileURI(directory.resolve("jvm.xml").toString());
Resource resource = resourceSet.createResource(uri);
resource.getContents().add(vm);
```

```
resource.save(null);                                                    ①
```

① A file per snapshot. The modules of the boot layer of a test run are a few dozen, and the file is small.

Loading it back needs the package registered, or a resource set configured from the module path, which registers it as a service:

Listing 4. Loading a snapshot

```
ResourceSet other = new ResourceSetImpl();
other.getResourceFactoryRegistry()
    .getExtensionToFactoryMap()
    .put("xml", new XMIResourceFactoryImpl());
other.getPackageRegistry().put(JvmPackage.eNS_URI, JvmPackage.eINSTANCE);  ①
JavaVirtualMachine loaded = (JavaVirtualMachine) other.getResource(uri, true)
        .getContents()
        .getFirst();
```

① The namespace URI in the file resolves to the generated package.

6.3. Asking a question

The question is ordinary code over the model:

Listing 5. Modules that read what they never required (`Snapshot.java`)

```
/**
 * Modules that read something they never required.
 */
public static Map<String, List<String>> readsWithoutRequires(JavaVirtualMachine vm) {
    Map<String, List<String>> answer = new TreeMap<>();
    for (Module module: vm.getModules()) {
        Set<Module> required = module.getRequires().stream()
                .map(Requires::getModule)
                .filter(Objects::nonNull)
                .collect(Collectors.toSet());
        List<String> implicit = module.getReads().stream()
                .filter(read -> read != module && !required.contains(read))
                .map(Module::getName)
                .sorted()
                .toList();
        if (!implicit.isEmpty()) {
            answer.put(module.getName(), implicit);
        }
    }
    return answer;
```

```
}
```

```
Map<String, List<String>> implicit = Snapshot.readsWithoutRequires(loaded);
```

For the boot layer of this module's own tests, the answer for the model is:

Module	Reads without requiring
org.nasdanika.sdk.runtime.models.jvm	org.eclipse.emf.common, org.eclipse.emf.ecore

That is `requires transitive` at work: the JVM model requires the core model, and the core model requires EMF transitively. In a larger assembly the answer gets more interesting. EMF and many other libraries are automatic modules, jars without a module descriptor, and the module system gives a module that reads one automatic module readability of all of them. In the assembly of an example in the Nasdanika SDK Runtime book, the core model, which requires only EMF and `java.scripting`, reads OpenTelemetry and a JSON library for exactly that reason. No build file says so, and a snapshot shows it in the first query anyone runs.

6.4. Types and methods, named the JVM's way

Types and members are named by binary names and descriptors, which reflection and `java.lang.invoke` compute:

Listing 6. A type and its public methods (`Snapshot.java`)

```
/**
 * A type and its public methods, identified by binary name and descriptor.
 */
public static Type type(Class<?> type) {
    Type ret = JVM.createType();
    ret.setName(type.getSimpleName());
    ret.setBinaryName(type.getName());                                    ①
    ret.setDescriptor(type.descriptorString());
    ret.setId("jvm:type/" + type.getName());
    ret.setKind(type.isInterface() ? TypeKind.INTERFACE : TypeKind.CLASS);
    int modifiers = type.getModifiers();
    ret.setAccess(Modifier.isPublic(modifiers) ? AccessLevel.PUBLIC : AccessLevel.PACKAGE);
    ret.setAbstract(Modifier.isAbstract(modifiers));
    for (java.lang.reflect.Method method: type.getDeclaredMethods()) {
        if (Modifier.isPublic(method.getModifiers())) {
            String descriptor = MethodType
                    .methodType(method.getReturnType(), method.getParameterTypes())
                    .toMethodDescriptorString();                          ②
```

```
            Method member = JVM.createMethod();
            member.setName(method.getName());
            member.setDescriptor(descriptor);
            member.setReturnDescriptor(method.getReturnType().descriptorString());
            member.setId("jvm:method/" + type.getName() + "#" + method.getName() + descriptor);
            member.setAccess(AccessLevel.PUBLIC);
            member.setStatic(Modifier.isStatic(method.getModifiers()));
            member.setAbstract(Modifier.isAbstract(method.getModifiers()));
            member.setDefault(method.isDefault());
            member.setBridge(method.isBridge());
            member.setSynthetic(method.isSynthetic());
            ret.getMethods().add(member);
        }
    }
    return ret;
}
```

① `Class.getName()` is the binary name: `java.util.Map$Entry` for a nested type, `[I` for an array of `int`.

② The method descriptor: parameter types in parentheses, then the return type.

And the identities are the ones every other tool uses:

Listing 7. Identity (`SnapshotTests.java`)

```
Type list = Snapshot.type(java.util.List.class);
assertThat(list.getId()).isEqualTo("jvm:type/java.util.List");
assertThat(list.getDescriptor()).isEqualTo("Ljava/util/List;");

Method get = list.getMethods().stream()
    .filter(m -> m.getName().equals("get"))
    .findFirst()
    .orElseThrow();
assertThat(get.getDescriptor()).isEqualTo("(I)Ljava/lang/Object;");      ①
assertThat(get.getId()).isEqualTo("jvm:method/java.util.List#get(I)Ljava/lang/Object;");
```

① Erased: `List.get` returns `Object` in the descriptor, whatever the element type.

Part III: Looking Ahead

Loaders, and what a model of the JVM is for once it has them.

Chapter 7. Looking ahead

Everything in this chapter is **PLANNED**.

7.1. A live loader

A live loader comes first, before any scanner of jars. It is smaller, with no traversal of jars and no decision about multi-release jars, and it produces a real graph to ask questions of: it walks the boot layer and its parents, reads each module descriptor, takes readability from the configuration, and reflects over loaded types. Where reflection does not reach, as for a field's constant value, it reads the loaded type's own class file through the JDK's class file API, final since Java 24, with no dependency.

7.2. A scanner of class files

A build-time scanner follows, and adds what a running VM cannot show: types that were never loaded. It populates the same model, and handles two things deliberately:

- **Multi-release jars**, recording which release it modeled.
- **Automatic modules**, whose names come from `Automatic-Module-Name` or the file name, not from a descriptor.

Loaders produce large models: a moderate application yields thousands of types and tens of thousands of members. That makes the model a useful stress case for anything that queries it, and a better corpus for a query engine than anything written by hand: realistic size, deep containment, heavy cross-referencing, and many instances of few classes.

7.3. Annotations, and the projection to Meta

Annotations are the most likely first addition to the model (Chapter 3). The projection to Meta gives anyone comparing a Java type with an Ecore class or a JSON Schema the route they expect, and parses generic signatures into type structure on the way.

7.4. Applications

Ask a running process what it is. Logs, metrics, traces and profiles describe behavior; a model of the JVM describes composition.

Deployed versus running. The difference between the dependency tree and the resolved graph, computed rather than argued.

Reports that are expensive today. Which class file versions are present, which modules are automatic, which packages are opened for deep reflection and to whom.

Version questions with an answer. With layers, which version of a module a given piece of code actually reads.

A join point for telemetry. Because identity is the JVM's own, a stack trace, a JFR event, a profiler's hot list and an OpenTelemetry span resolve to model elements without a mapping. Static analysis cannot see reflective calls, and the run time cannot see what was never exercised, so the union is a call graph with evidence rather than inference.

Coverage that knows what runs. With a live loader recording what was loaded and coverage attached from builds, two lists appear that a coverage tool alone cannot produce: covered but never loaded, which is tests for code that does not run, and loaded but not covered, which is the real list of risks.

The JVM as files. A snapshot presented as a volume of the runtime and served over HTTP or WebDAV is a running system that an agent reads with the file tools it already has.

7.5. Documentation, and a licensing constraint

Documentation of the model's elements is meant to be generated, and a constraint applies to every element. The Java SE specifications are under Oracle's terms, and OpenJDK's Javadoc is under the GPL version 2 with the Classpath Exception; neither permits publishing derived text under the runtime's license.

That does not block generated documentation, because concepts are not copyrightable and expression is. It does mean that an assistant asked to describe `requires transitive` may reproduce Javadoc it remembers. Generated documentation must be original prose, never

pasted, and checked for verbatim overlap with the official text before it is published.

Appendix A: Reference

A.1. Classes

Class	What it is
`JvmElement`	Abstract base of every element but the root; a core model `NamedElement`
`JavaVirtualMachine`	The root: one JVM, live or scanned
`Layer`	A module layer
`ClassLoader`	A class loader, with its unnamed module
`Module`	A named or unnamed module, its directives, and the modules it reads
`Requires`, `Exports`, `Opens`, `Provides`	The directives of a module
`Package`	A package, containing its types
`Type`	A class, interface, enum, record, annotation, primitive or array
`Member`	Abstract base of methods and fields
`Method`	A method, a constructor (`<init>`) or a static initializer (`<clinit>`)
`Field`	A field, with its constant value
`Parameter`	A parameter, identified by its index

A.2. Enumerations

Enumeration	Literals
`ModuleModifier`	`OPEN`, `AUTOMATIC`, `SYNTHETIC`, `MANDATED`
`RequiresModifier`	`TRANSITIVE`, `STATIC`, `SYNTHETIC`, `MANDATED`
`DirectiveModifier`	`SYNTHETIC`, `MANDATED`
`TypeKind`	`CLASS`, `INTERFACE`, `ENUM`, `RECORD`, `ANNOTATION`, `PRIMITIVE`, `ARRAY`
`AccessLevel`	`PUBLIC`, `PROTECTED`, `PACKAGE`, `PRIVATE`

A.3. Identity

Element	URI	Example
Module	`jvm:module/<module name>`	`jvm:module/java.base`
Package	`jvm:package/<binary package name>`	`jvm:package/java.util`
Type	`jvm:type/<binary name>`	`jvm:type/java.util.Map$Entry`
Method	`jvm:method/<owner binary name>#<name><descriptor>`	`jvm:method/java.util.List#get(I)Ljava/lang/Object;`
Field	`jvm:field/<owner binary name>#<name>:<descriptor>`	`jvm:field/java.lang.Integer#MAX_VALUE:I`
Parameter	`jvm:parameter/<owner method>#<index>`	`jvm:parameter/java.util.List#get(I)Ljava/lang/Object;#0`

Appendix B: Building this book

This book is AsciiDoc in `models/jvm/model/src/docs/asciidoc`, built with the model after its tests have passed:

```
mvn -pl models/jvm/model -am verify                          # the model, its tests and this book
mvn -pl models/jvm/model verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/jvm/model verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/models/jvm/tests/book`, and of the module declaration. A listing line holds at most 96 characters, and a warning fails the build.

The answer in Chapter 6 is what `SnapshotTests` prints for the boot layer of the module's tests. A change to the module's dependencies changes it; run the tests and update the table.

Appendix C: Glossary

Automatic module

A jar without a module descriptor on the module path. Its name comes from `Automatic-Module-Name` in its manifest, or from the file name. A module that reads one automatic module reads all of them.

Binary name

The name of a type as the JVM knows it: `java.util.Map$Entry` for a nested type, `[I` for an array of `int`.

Class file version

The major version in a class file's header, such as 69 for Java 25.

Descriptor

The JVM's erased form of a type, `Ljava/lang/String;`, or of a method's parameters and return type, `(I)Ljava/lang/Object;`.

Layer

A module layer: a set of resolved modules and the class loaders that serve them. The boot layer is the first.

Readability

The relation "module A can see the public types of the packages that module B exports to it", computed by resolution from `requires`, `requires transitive`, service binding and automatic modules.

Signature

The class file attribute that records generic types, such as `<T:Ljava/lang/Object;>(TT;)Ljava/util/Optional<TT;>;`. Data, not identity.

Unnamed module

The module of everything a class loader loads from the class path. Every class loader has one.